Ninaivu

நினைவு · memory

For whoever looks after the technical side.

Technical administrator guide · version 1.10.8

Docker
on a NAS

Source
a checkout

Command line
flags and tools

Contents

Everything in this guide, and the page it is on.

Technical administrator guide 3

This page is for whoever looks after the technical side of Ninaivu: the source code, Docker, running from a checkout, the command line,...

- Where things are 3
- More on the installers 3
- Docker 5
- From a checkout 5
- Running it from the command line 6
- Reaching Ninaivu from outside 8
- The TV album 9
- Tuning to the machine 10
- Updating to a new version 10
- Environment variables and the state folder 12
- Further reading 12

NINAIVU 1.10.8

Technical administrator guide

This page is for whoever looks after the technical side of Ninaivu: the source code, Docker, running from a checkout, the command line, environment variables and where to report a problem. The guide tells a household how to install Ninaivu with the installer for their computer and use it. This page is in English only.

Where things are

Source code	https://github.com/javajaga-usa/Ninaivu
Installers (the download page the guide points to)	https://github.com/javajaga-usa/Ninaivu/releases/latest
Reporting a problem	https://github.com/javajaga-usa/Ninaivu/issues
Extensions (Creative Studio, Gemini and the rest)	https://github.com/javajaga-usa/Ninaivu/tree/main/extensions
The docs site	https://javajaga-usa.github.io/Ninaivu/

When reporting a problem, say which version (**System → Home & extensions** shows it under *This home*), what was done, and what the log said.

More on the installers

- **Windows.** Once Ninaivu is in the winget catalogue, `winget install Ninaivu.Ninaivu` will work too.

- **macOS.** Once Ninaivu is in Homebrew, `brew install --cask ninaivu` will work too. `open -a Ninaivu --args --tray` opens the menu-bar tray instead of the Control Panel.
- **Linux and Raspberry Pi.** `--photos DIR`, `--prefix DIR`, `--no-service` and `--quiet` skip the installer's questions, for a headless Pi set up over SSH. Running it again upgrades: the photographs folder the service was given is kept (pass `--photos` to change it), and so are the index, the settings and the AI models (`~/.local/state/ninaivu/ai-models`). The service is told its state folder (`NINAIVU_STATE_DIR`); an install from 1.0 whose data is in `~/.ninaivu` keeps using it there. `~/.local/share/ninaivu/uninstall` removes the program, the commands, the menu entry and the service, and only those: a `--prefix` folder with other things in it is left with them. `uninstall --purge` also removes the state folder the server used and the AI models. The library is never touched.

The Linux system service

`sudo sh Ninaivu-<version>-linux-arm64.sh` installs under `/opt/ninaivu` instead, with a system service that runs as its own user, `ninaivu`, not as root: a fault in reading a photograph or a video is then that user's, not the whole machine's. It may still use port 80, and the service has systemd's hardening (no new privileges, `/usr`, `/boot` and `/etc` read-only, a private `/tmp`). Its state is in `/var/lib/ninaivu`, and the default photographs folder is `/var/lib/ninaivu/photos`. A folder elsewhere has to be readable and writable by that user; the installer says so when it is not, for example `sudo setfacl -R -m u:ninaivu:rwX -m d:u:ninaivu:rwX /srv/photos`. Run the maintenance commands as that user too, so the files they write stay its own: `sudo -u ninaivu ninaivu reset-password`.

`sudo systemctl stop ninaivu` lets Ninaivu finish what is in hand: the service allows 45 seconds (`TimeoutStopSec`), and the log names anything that had not stopped by then.

An install from 1.0 ran the service as root, with its data in `/root/.ninaivu`. An upgrade leaves it so, and says so. To move it to its own user: stop it (`sudo systemctl stop ninaivu`), move the data (`sudo mv /root/.ninaivu/* /var/lib/ninaivu/`), give the user the data and the library (`sudo chown -R ninaivu: /var/lib/ninaivu`, and access to the library as above), delete `/etc/systemd/system/ninaivu.service`, and run the installer again with `--photos` and the library folder.

Docker

For a NAS or a Linux box:

```
git clone https://github.com/javajaga-usa/Ninaivu.git
cd Ninaivu
cp installers/docker/.env.example installers/docker/.env   # set MEDIA_DIR to your
photo folder
docker compose -f installers/docker/docker-compose.yml up -d
```

The family app answers on port 5000 to the whole network; the console on 3000, published on the NAS itself only (`127.0.0.1:3000`). To make the administrator from another computer, open the console through an SSH tunnel or on the NAS, and give the **setup code** printed in `docker logs ninaivu`. The `.env` file is only for folders and ports: everything about how Ninaivu behaves is chosen in the console, and nothing in `.env` undoes it at the next restart. The image leaves out search by description unless built with `--build-arg WITH_AI=1`. Compose builds the image from the checkout; each release also publishes one, `ghcr.io/javajaga-usa/ninaivu:<version>` (and `:latest`), to use in `docker-compose.yml` instead of building. `docker stop` and `docker compose down` give it 45 seconds to finish what is in hand (`stop_grace_period`), rather than Docker's ten. The operations guide covers systemd, reverse proxies, HTTPS and large libraries.

From a checkout

Running from the source needs Python 3.12 or newer (the installers bring their own).

```
git clone https://github.com/javajaga-usa/Ninaivu.git
cd Ninaivu
start.cmd ~/Pictures                # Windows: or double-click start.cmd
sh launcher/start.sh ~/Pictures     # macOS and Linux
```

The launcher makes a virtual environment, installs what is missing (nothing large: search by description is added later, if you want it), serves HTTPS on port 443 — or 8443 on Linux, where an ordinary user may not use 443, the same port every time — finds free ports and opens your browser. `python -m ninaivu <folder>` runs the server directly, on plain HTTP and port 80 (8080 without the right to use 80), unless given `--https`.

On Windows, `start.cmd` needs no Python beforehand: when it finds no Python 3.12 or newer, it installs Python 3.12 for the current user first (with winget, or from python.org when winget is missing), then carries on. No administrator rights are needed. **Start - Ninaivu Control Panel.vbs** opens the Control Panel once `start.cmd` has run.

On a Mac, double-click **Setup Ninaivu.command** in the Ninaivu folder once instead of using a terminal. It clears the download quarantine, installs Python 3.12 if the Mac has no Python 3.12 or newer (after asking: with Homebrew when the Mac has it, otherwise the installer from python.org, which is run only when it carries the Python Software Foundation's signature, and for which macOS asks your password), installs what Ninaivu needs, and puts two apps in Applications: **Ninaivu**, which starts the library in a Terminal window and asks once which folder holds your photos, and the **Ninaivu Control Panel**, which starts and stops it without one. `sh launcher/start.sh` offers the same Python install. More on the Control Panel.

A checkout keeps its log in `.ninaivu-control/server.log` beside it.

Videos for guests and share links need ffmpeg. A phone writes the place a video was filmed into the file, and guests, share links and family members whose copies leave out the home location are given a copy without it, made by ffmpeg. Without ffmpeg on the computer Ninaivu runs on, those people are told the video cannot be shown rather than sent the original. The Docker image includes it; elsewhere, install ffmpeg from the system's packages (on Linux) or Homebrew (on a Mac).

Running it from the command line

`python -m ninaivu <folder>` (from a checkout) starts the server, and these are the flags worth knowing:

FLAG	
`--admin USER:PASSWORD`	make the administrator without a browser
`--port`, `--admin-port`	the two ports (80 — 443 with HTTPS — and 3000 by default)
`--local-only`	this computer only; phones cannot reach it
`--https`, `--cert`, `--key`	HTTPS with Ninaivu's own certificate, or yours
`--ai auto|clip|light|off`	the image model; *auto* follows the hardware tier
`--faces`, `--ocr`, `--places`	turn the passes on from the start
`--no-watch`	do not watch the folders for changes
`--open-browsing` / `--private`	whether visitors may look without signing in
`--lock-roots`	confine the folder picker to the library folders

FLAG	
`--admin-host 0.0.0.0`	open the console to the network from the start
`--strict-port`	refuse to start rather than move to another port
`--name`, `--no-mdns`	the name on the home network (`ninaivu.local`), or none
`--map-tiles`	OpenStreetMap tiles on the map (they show roughly where photographs were taken)

When the port is taken by another program, Ninaivu moves to the next free one and says so; `--port` chooses another, `--strict-port` refuses to move.

And the maintenance commands, which work in an installed copy too (the installers put a `ninaivu` command on the computer):

COMMAND	
`ninaivu backup --out <folder>`	a copy of the index, settings and certificate
`ninaivu list-backups <folder>`	the copies in a folder
`ninaivu restore <file>`	put one back (Ninaivu stopped)
`ninaivu reroot --from <old> --to <new>`	tell the index the library moved
`ninaivu reset-password <name>`	a new password for someone, from this computer (a forgotten administrator password); the Control Panel's **Reset a password…** does the same
`ninaivu import-lite <file>`	bring a Ninaivu Lite household across: people, who sees what, favourites, albums, share links (Ninaivu stopped, after its first scan)
`ninaivu offsite-restore --recovery <file> <copy> <output>`	put the photographs back from an off-site copy, on any computer

The storage check reads the whole library and takes hours on a large one. Until the console has a button for it, `POST /api/admin/scrubber/stop` (signed in to the console) stops it after the file it is reading; a restart does not start it again, and Start carries on from where it stopped.

Reaching Ninaivu from outside

Ninaivu decides for every request whether it came from home or from the internet. Home is the house's own network (IPv6 included), this computer, Tailscale and WireGuard. Any public address counts as the internet, and so do a forwarded router port, a tunnel or proxy Ninaivu was not told about (forwarded headers from a peer not in `trusted_proxies` and `NINAIVU_TRUSTED_PROXY_ADDRESSES`), and Tailscale Funnel. From the internet:

- profiles set to *tap to enter* do not open, and nobody can browse without signing in;
- plain HTTP straight from a public address is refused, on every path;
- the console refuses every request, whatever `--admin-host` or *Open this console from other devices* say, unless `console_from_internet` (**Advanced settings → Remote access**, off by default) is turned on for a tunnel or proxy on purpose;
- 20 wrong administrator passwords in half an hour pause sign-in by username for 30 minutes, then an hour, then two; the computer Ninaivu runs on can still sign in.

Everywhere, uploads are refused while the server's disk would be left with under 2 GB free (phone backups already were), an upload that is really a playlist (`#EXTM3U`, an ffconcat list) is refused, and signing out over HTTPS empties the browser's cache of thumbnails.

What the household should set up, once:

1. Reach Ninaivu from away through **Tailscale** (or WireGuard). Forward no port on the router, switch UPnP off there, and keep inbound IPv6 blocked (most routers' default). Never turn on Tailscale Funnel for Ninaivu.
2. Turn on **HTTPS certificates** in Tailscale's admin page, so Ninaivu serves its `.ts.net` name with a certificate phones accept (step by step).
3. Turn on the computer's **firewall**, allowing Ninaivu, and leave *Open this console from other devices* off unless it is needed.
4. A **long administrator password** (14 characters or more, or four random words), kept in a password manager and used nowhere else.
5. A **PIN on every profile**, and open browsing off if anyone not fully trusted uses the Wi-Fi.
6. **Disk encryption** on the computer (FileVault, BitLocker or LUKS): Ninaivu does not encrypt the library or the index on this computer; only the cloud and off-site copies are encrypted.
7. **Two-step sign-in** on the accounts around Ninaivu: Tailscale, Google Drive, the off-site storage service, GitHub.
8. **Keep things up to date**: the operating system, Tailscale, ffmpeg, and Ninaivu when a release comes out (it never updates itself).
9. On a fork of the repository, GitHub's **Dependabot security updates**.

Ninaivu itself has no two-step sign-in yet, and HTTPS is not on by default on the home network; reach it by its Tailscale name at home too, or start it with `--https`, if anyone untrusted may be on the Wi-Fi.

The TV album

The TV album puts one album on the televisions at home, with no app to install. Ninaivu answers as a DLNA media server, which is what a smart TV's own media player (often called *Media*, *Media Player* or the TV's source list), VLC (*Local network → Universal Plug'n'Play*) and Kodi (*Videos* or *Pictures → Add source → UPnP devices*) look for. The TV shows it as "*home name* TV album", and the remote walks through the photographs and videos, newest first.

It is off until an administrator chooses the album, in the console under **System → Server → TV album**: pick the album (or *Off*), change the port if 8200 is taken, and press **Save**. The block then says whether it is showing, how many photographs and videos the TV sees, and lists the addresses a TV can be given by hand when it does not find the server by itself. Behind it are the settings `tv_album` (the album's number; 0 is off) and `tv_port`, and `GET` / `PUT /api/admin/tv-album` for scripts.

What the TV sees is kept narrow, because DLNA has no sign-in and anything on the home network can ask:

- only the chosen album, and of it only the photographs and videos a family member could see: never anything hidden, flagged, in the bin, outside the family's date limit, or a sound recording, even when the album holds one;
- only on the home network: a device with a public address, or one reaching Ninaivu over Tailscale or WireGuard, is refused, and nothing is answered with *Network access* off;
- every address starts with a random secret, so a guessed address finds nothing. **New addresses** in the same block makes a new one, and every address handed out before stops working; the TVs find the new one by themselves the next time they look for media servers.

Photographs are sent as the same upright copy, at most 2560 pixels and with no location or camera details in it, that a share link sends. Videos are sent as they are. The firewall must let the TV reach `tv_port` (TCP) and UDP port 1900, which is where TVs look for media servers; if 1900 cannot be had, the TV album still runs and the TV can be given its address by hand.

Tuning to the machine

At every start Ninaivu measures the computer (its cores, memory, graphics processor, whether it is a Raspberry Pi or another single-board computer, and whether the library is on a spinning disk) and picks a profile: **Small box** (a Pi, under 6 GB of memory, or two cores), **Everyday computer**, or **Powerful computer** (eight cores and 16 GB, or a graphics processor and 16 GB). The profile sizes the indexing workers, the analysis threads, the web threads, the image model's batch, the moments described per video, the backup uploads at once and the index cache. **Peak performance** is only ever chosen by hand (or by the desktop panel's Performance mode): every core (18 of 18) and up to 95% of the memory, the rest of the memory left for the operating system. The desktop panel's Standard mode is **Everyday computer**, half the cores (9 of 18). **Powerful computer** uses 80% of the cores and keeps at least two back (14 of 18). Every core the system reports is counted, of every kind: an Apple chip's Super, Performance and Efficiency cores alike. Where the orientation survey runs beside the scan (a graphics processor and a solid-state library), its readers are part of the same share: Peak keeps two cores for it. A profile chosen on the Tuning page wins over the panel's mode, so set it back to automatic there for Performance mode to take effect.

The console's **System → Tuning** page shows what was measured, the profile, what the numbers are expected to use, and every number with where it came from. Any of them can be set outright and put back to automatic; most apply at once, the indexing workers at the next scan, and the web threads and the index cache after a restart, which the page offers. A number given at start (`--workers`, `NINAIVU_SERVER_THREADS`, `NINAIVU_COMPUTE_THREADS`, or written into `config.json` by hand) is kept unless it is set on the page. The choices are saved in `config.json` as `tuning_profile` and `tuning`.

Updating to a new version

An update replaces the program and nothing else. The photographs are never read or written by an installer, and neither is the state folder (the index, `config.json`, the thumbnails, `setup-code.txt`, the keys) or the AI models.

Ninaivu never looks on the internet for a newer version, and nothing is ever downloaded or installed by Ninaivu itself. An update is a newer installer someone downloads from the releases page and runs, when they choose to.

Stop Ninaivu before updating. Press **Stop** in the Control Panel (or the console's **Server** page, or the tray) and wait until it says Stopped, then close the Control Panel and the tray; for the Linux service, `systemctl stop ninaivu`; for Docker, the `pull` and `up -d` below do it. A server left running from the old version would keep running from files being replaced under it. The Windows installer says so when it finds an earlier version, asks a running one to stop properly, and waits

with **Retry** while it is still in use; the Linux installer asks a running one to stop properly and leaves everything as it was if it will not. A Mac cannot tell, so on a Mac stop Ninaivu before dragging the new app over the old one (the disk image has a *Before updating* note saying so).

HOW IT WAS INSTALLED	HOW TO UPDATE	WHAT IS REPLACED	WHAT STAYS AS IT WAS
Windows installer	run the newer `.exe`	`Python`, `pkgs` and `bin` in the install folder, after Ninaivu is stopped	the state folder `%USERPROFILE%\.ninaivu`, the models in `%LOCALAPPDATA%\Ninaivu\ai-models`, `.ninaivu-control`, the library
Windows portable zip	stop Ninaivu, extract the newer zip into a new folder, and move `app\data` and `app\ai-models` from the old folder into the new `app` folder	everything in the zip (`Ninaivu.exe`, `app\python`, `app\pkgs`)	`app\data` (index, settings, people), `app\ai-models`, `logs`, the library
macOS app	drag the new **Ninaivu** over the old one in Applications (or `brew upgrade --cask ninaivu`)	`Ninaivu.app`	`~/.ninaivu`, `~/Library/Application Support/Ninaivu`, the library
Linux and Raspberry Pi	run the newer `.sh` the same way	the private Python, the commands, the menu entry, the service file (with the same photographs and state folders in it)	the state folder, `ai-models`, the library
Docker	`docker compose pull` (or `build`), then `up -d`	the image	the `ninaivu_state` volume and `MEDIA_DIR`
A checkout	`git pull`, then start it again	the source	the state folder, `.ai-models`, the library

In Docker, never add `-v` to `docker compose down`: that deletes the state volume, which is the index. The library itself is still untouched.

The first start of a new version is the only time anything of the old setup changes: it adds what it needs to `index.db` and `archive.db`, in place. Before that, it copies the whole state folder into `<state>/backups/before-update` (the same verified bundle the scheduled backup makes, the three most recent kept) and notes the version in `<state>/version.json`. If the copy cannot be made (a full disk) Ninaivu does not start, and says so; `NINAIVU_SKIP_UPDATE_BACKUP=1` starts it once without the copy, for someone who has a backup of their own.

Going back to an older version. An older Ninaivu refuses to start on an index a newer one has changed in a way it does not know, and names the copy to restore; it never writes over it. To go back: install the older version, then `ninaivu list-backups "<state>/backups/before-update"` and `ninaivu restore <bundle>` (with Ninaivu stopped), which puts back the state from just before the update. Anything done after the update (new albums, names) is not in that copy; the photographs are not affected either way. Versions up to 1.0.4 do not have this check, so going back to one of those needs the restore first.

Environment variables and the state folder

Some settings also read an environment variable (`NINAIVU_STATE_DIR`, `NINAIVU_AI_ENGINE`, `NINAIVU_HARDWARE_TIER`, `NINAIVU_FACES`, …; the full list is in `ninaivu/server/config.py`). Folders and ports from the environment always apply; anything about behaviour only fills in what nobody chose in the console. A number out of range, from the environment or typed into `config.json` by hand, is brought within its limits (one from the environment is named in the log); a value of the wrong kind in `config.json` is set aside with a message and Ninaivu starts anyway. The state directory (`~/.ninaivu`, or `$XDG_DATA_HOME/ninaivu`, or `NINAIVU_STATE_DIR`) holds the index, the thumbnails and `config.json`. The AI models live in `.ai-models` in a checkout; an installed copy keeps them in a folder of the user's own, which an upgrade does not replace (`%LOCALAPPDATA%\Ninaivu\ai-models` on Windows, `~/Library/Application Support/Ninaivu/ai-models` on a Mac, `~/.local/state/ninaivu/ai-models` on Linux). `NINAIVU_AI_MODELS_DIR` or the `ai_models_dir` setting moves them; the Docker image sets the former to `/data/state/ai-models`, on the state volume.

Further reading

- Operations and production — Docker, systemd, reverse proxies, large libraries, backups.
- The tray, the Control Panel and HTTPS
- Backup and recovery and encryption and the recovery file
- Moving to another machine
- Contributing and security



Ninaivu

For whoever looks after the technical side.

THE GUIDE, ONLINE
javajaga-usa.github.io/Ninaivu/

SOURCE
github.com/javajaga-usa/Ninaivu

REPORT A PROBLEM
github.com/javajaga-usa/Ninaivu/issues

Ninaivu 1.10.8 · MIT licence